课程主页

QQ群

Overview

Introduction to Computer Systems

https://xjtu-ics.github.io/

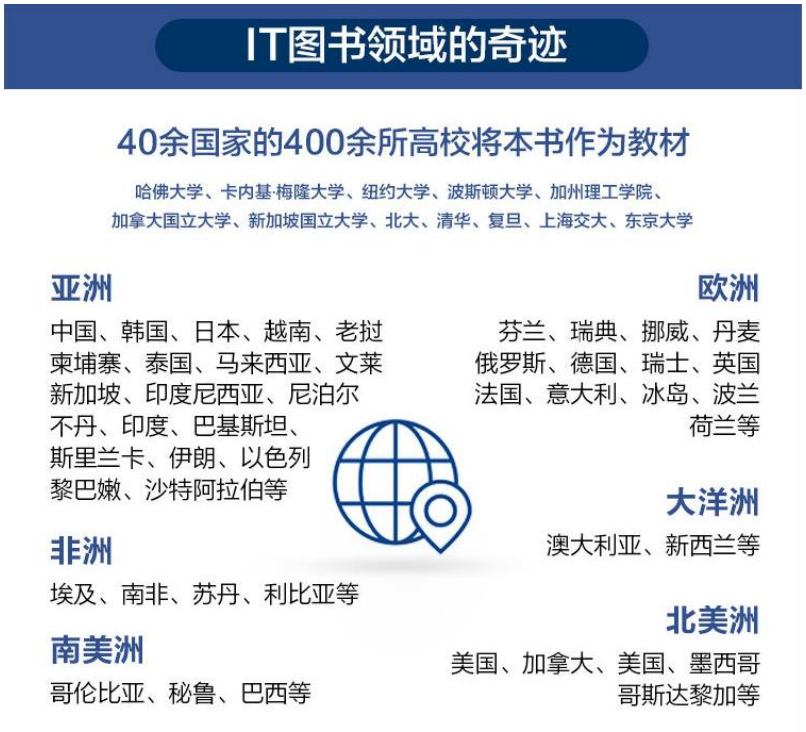
2026. Spring

Xi’an Jiaotong University

Danfeng Shan

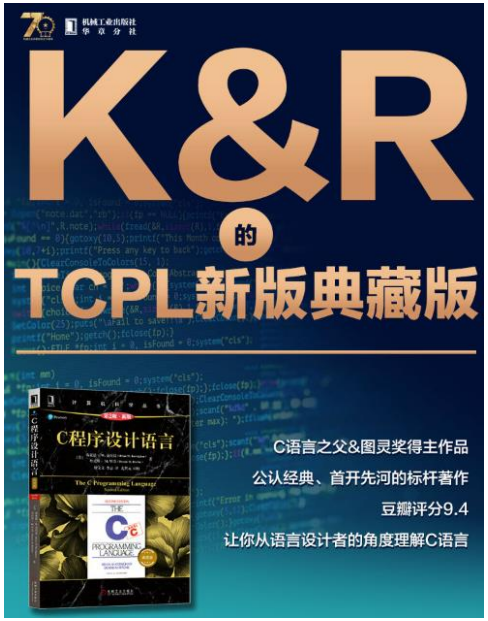
教材与参考书

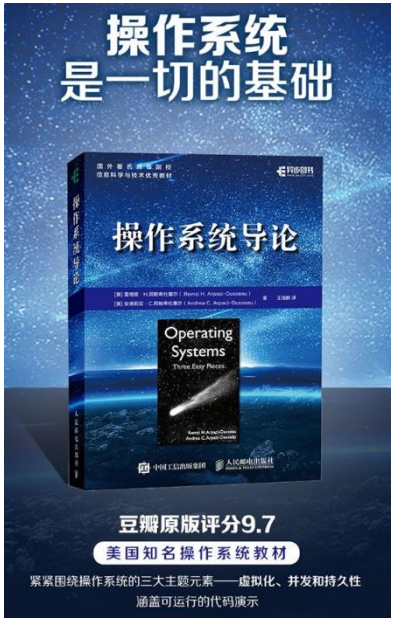
- 深入理解计算机系统（第三版）
- Computer Systems: A Programmer’s Perspective (CSAPP 3e)



参考书

- C语言程序设计 第二版 （K&R）
- 操作系统导论（OSTEP）





这门课
不一样！

- 来自学生的评价
  - 非常好的课程，使我有一种不是在西交读cs的感觉
  - lab 才是这门课的精华
  - 虽然对我来说实验有一定的难度，但做起来真的很爽，希望课程越来越好
  - 实验不要抄袭，真给0分

- 来自学生的误解
  - 简单的导论课程
  - 可以刷分课程

教学规划——授课

内容	参考学时
信息的处理与表示	4
程序的机器级表示	16
存储器体系结构	4
处理器体系结构	8
程序优化	4
程序链接	4
异常控制流	6
虚拟存储器	10
网络编程	4
并行编程	4

教学规划——实践

内容	参考学时
datalab	2
bomblab	4
attacklab	4
cachelab	4
optlab	4
linkerlab	4
malloclab	4

考核方法

- 课堂表现 10%
  - 考勤、课堂纪律、上课回答问题

- 项目实践 50%
  - Auto-Grading系统对代码自动打分
  - Anti-Cheating系统自动检测代码抄袭
  - 抄袭/作弊是高压线，一经核实双方项目实践分数为0

- 💥💥 期末*机试* 40%💥💥
  - 以课堂和Lab内容为主

关于实验

- 实验环境：Linux/GCC
  - 提供完整环境配置的服务器（ICSServer）
  - 提供预先写好的Makefile
  - 组织Tutorial，带新人快速上手
- 实验发布：课程主页
  - https://xjtu-ics.github.io/
  - 有详细的实验指导书
- 实验提交和分数公布：思源学堂
  - https://lms.xjtu.edu.cn/course/28468
  - 允许迟交，但会根据延后时间扣分

关于实验

- 做好时间管理
  - ✗ 跟不上节奏、在截止日期前疯狂弥补☹
  - ✓ 紧跟节奏😄
  - 预期：每个lab需要2周的时间全力以赴完成
- 实验所需预备知识

必要	有益
• C编程 • Linux命令行 • ssh • gcc/gdb	• vim • Make/Cmake • Git • Google

🚫 严禁作弊

- 本课程所有实验要求独立完成
- 抄袭/作弊 = 使用他人的成果/信息，不以“有没有复制粘贴”为唯一标准
- 我们的目标不是“抓人”，而是确保大家学到该学到的能力，并保证成绩公平
- 交上来的必须是你自己能解释、能复现、能维护的东西

什么算抄袭/作弊

- 以下都属于使用他人成果/信息（即抄袭/作弊）：
  - 借用代码：复制粘贴、照着抄、重新敲一遍、对着别人的文件/屏幕写
    - 改变量名/改注释/调格式 不等于原创
  - 口头/文字“讲解到代码级别”：把关键实现细节、具体写法、函数结构讲给别人
    - 即使你们“只是讨论怎么把几段代码拼起来”，也可能越界
  - 网上搜现成答案/题解/仓库：GitHub、博客、CSDN、论坛、QQ群文件等
  - 用往届/别的课/网上的同类作业代码，或“借鉴到可以对上结构”
  - 使用AI工具（ChatGPT/Copilot）生成/美化代码

AI 工具相关

- 禁止使用任何 AI 工具
  - 生成作业/实验代码（含补全、重写、改写、优化）
  - 针对你作业代码进行定位 bug / 给出修改方案

- 禁止把你的作业代码（哪怕是一部分）上传/粘贴到任何 AI 工具
  - 云端/本地/IDE 插件都算（例如：ChatGPT、Copilot、通义、Kimi、Claude 等）

- 请在完成作业/实验时关闭 IDE 的 AI 补全/自动建议（如 Copilot 类功能）

AI往往会“吐出”高度相似的实现，最终导致多人提交雷同代码，风险极高

“提供帮助”也可能违规

- 以下属于违规提供帮助（同样违规）：
  - 把你的作业文件/截图/仓库链接发给别人
  - 让别人可以访问你的代码
    - 公共仓库、未设权限的仓库/网盘/共享目录
    - 电脑未锁屏、把代码投屏给别人拍照
  - 帮别人改代码、对着代码逐行讲、直接给可运行实现
  - 在网上售卖课程代码

- 提醒：借的人违规，给的人也违规。“我只是好心”不是免责理由。

什么是允许的讨论/参考

- 允许且鼓励：
  - 讨论高层思路：模块如何划分、接口怎么设计、测试怎么写（只到“原则/策略”）
  - 问工具用法：编译、调试器、版本控制、性能分析工具怎么用
  - 查资料：教材、官方文档、man page、标准库说明、公开的通用知识
    - 例如：查 malloc/free 的规范、gdb 的命令
    - 不要搜索“本课程/本作业关键词 + solution/题解/源码”
  - 向助教/老师求助：可以展示你的代码，让我们教你怎么调试、怎么定位

严禁作弊

🚫 严禁作弊

- 其他例子
  - “我没复制，我是照着同学代码重新敲的”
  - “我只是看了一眼结构/核心函数怎么写”
  - “我只问了 AI：这段报错怎么改/怎么实现这个函数”
  - “我从网上找了类似题，改了改就提交”
  - “我把仓库设成 public/把代码发群里，别人拿去用我不管”

- 不算作弊（在不涉及代码细节的前提下）：
  - 讨论：思路是否用分治/缓存/并发
  - 问：某个系统调用/库函数的正确用法（并自己实现作业要求）
  - 让 TA 看你的实现并指导你如何定位问题（不是给你最终答案）

为什么我们这么严格 & 违规后果

- 为什么严格
  - 这门课的能力 只能靠自己动手踩坑学会：设计、调试、验证、复盘
  - 你需要学的是“找到答案的过程”，不是“把答案交上去”
  - **公平：***你的努力不应该被别人的投机稀释*

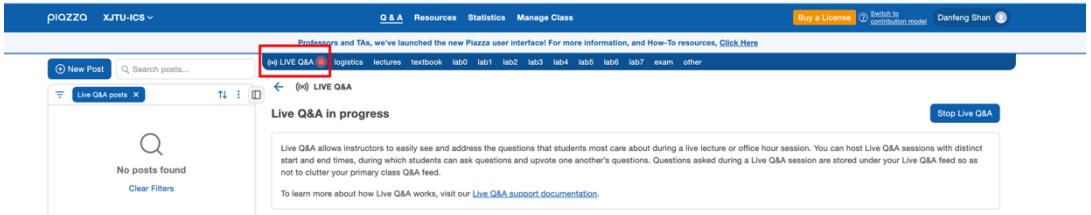
- 违规后果
  - （双方）实验记0分

🚫 严禁作弊

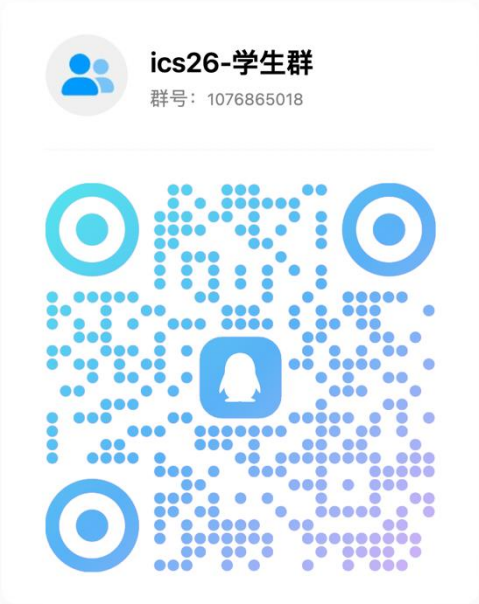
“竞技体育，成绩是练出来的，冠军是撞出来的。”

答疑

- Piazza：一款专业的国际性课程答疑论坛
  - 注册链接：https://piazza.com/stu.xjtu.edu.cn/spring2026/xjtuics
- 一对一：Office Hour（具体请查看课程主页）
  - 前两周：每天晚上18:00-22:00 + 周末12:00-22:00
  - 后六周：每天晚上20:00-22:00



- QQ群：不保证每个问题都能被解答
- 《提问的智慧》



扫一扫二维码，加入群聊

- 遇到问题不要不解决！

Course Overview

What you have known

Write a program

How programs are executed?

Details of each component

What you are about to learn

Why Systems Matter in the AI Era?

- From the perspective of business

OpenAI被曝正寻求千亿美元融资，单季亏超120亿美元，公司进入「红色代码」紧急状态，透露哪些信息？

每日经济新闻，已认证机构号

OpenAI与Anthropic毛利目标双双落空，非付费用户与算力成主要负担

李佳 发表于 2026年02月25日 23:11

摘要：OpenAI与Anthropic因推理成本失控正面临严峻盈利考验。去年OpenAI毛利率从40%降至33%，远低于预期，其推理成本因服务需求超预期激增至84亿美元。免费用户占比95%及高算力产品进一步拖累利润。尽管服务付费用户效率有所提升，两家公司要实现本十年末60%以上的毛利率目标仍面临巨大挑战。

新浪科技

英伟达AI霸权稳了！ Q4营收681亿美元 同比增长73% 净利润翻倍

快科技官方 原创作者 02.26 07:53

博通Q3 AI芯片收入超预期增63%，神秘新客户下单百亿，料下财年AI前景“大幅”改善 | 财报见闻

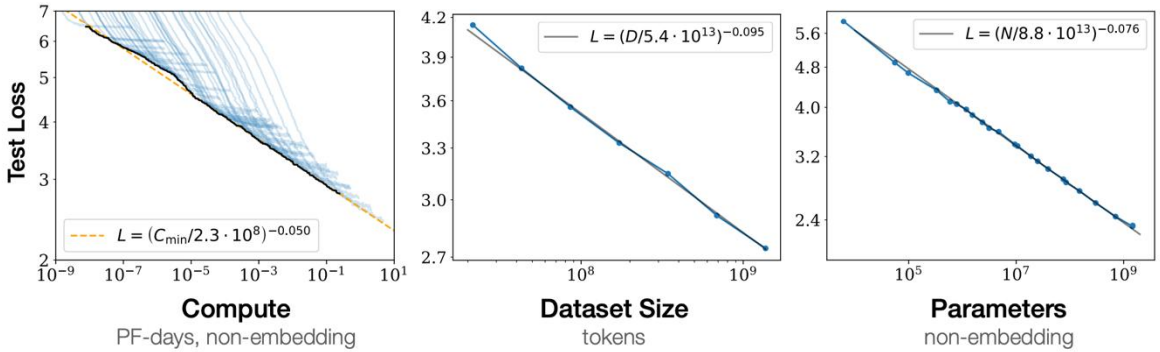
李丹 2025-09-05 06:48

Why Systems Matter in the AI Era?

- From the perspective of technology

“一盒内存条堪比上海一套房”，内存条为何成“抢手货”？

2026-02-28 15:52 发布于：湖北省



“Performance depends strongly on scale, weakly on model shape” [1]

“You can’t scale by throwing hardware at the problem forever.” --- Lidong Zhou, Microsoft

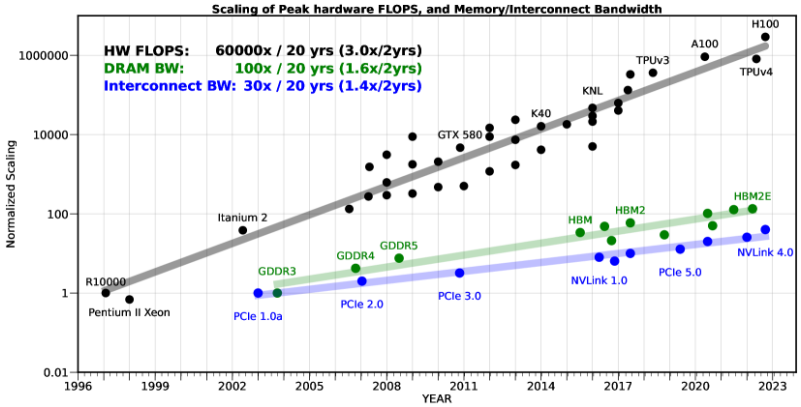
[1] Kaplan J, McCandlish S, Henighan T, et al. Scaling laws for neural language models[J]. arXiv preprint arXiv:2001.08361, 2020.
[2] Gholami A, Yao Z, Kim S, et al. Ai and memory wall[J]. IEEE Micro, 2024, 44(3): 33-39.

Why Systems Matter in the AI Era?

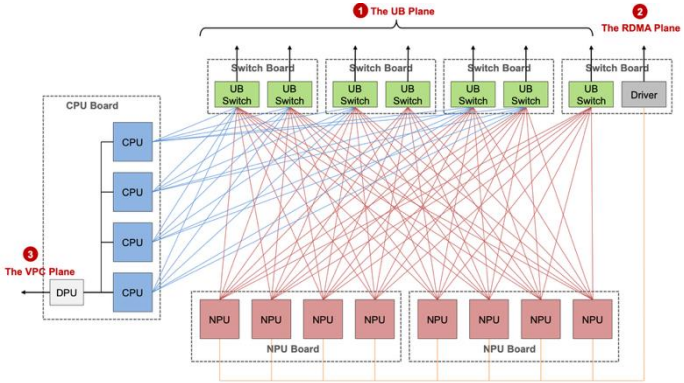
- From the perspective of technology

“一盒内存条堪比上海一套房”，内存条为何成“抢手货”？

2026-02-28 15:52 发布于：湖北省



“memory, rather than compute, the primary bottleneck in AI applications” [2]



CloudMatrix 384: 384 NPUs, 192 CPUs into a unified supernode, interconnected via an Unified Bus Network

[1] Kaplan J, McCandlish S, Henighan T, et al. Scaling laws for neural language models[J]. arXiv preprint arXiv:2001.08361, 2020.
[2] Gholami A, Yao Z, Kim S, et al. Ai and memory wall[J]. IEEE Micro, 2024, 44(3): 33-39.

Why Systems Matter in the AI Era?

- From the perspective of programmer
  - Vibe Coding is All You Need? “Code is cheap. Show me the talk.”
  - A fun story (made-up from Cloudflare outage on November 18, 2025)
    - A guy gets hired because he’s great at vibe coding.
    - He doesn’t write code; he writes prompts.
    - One day he ships an LLM-generated patch to production and—surprise—Cloudflare goes down.
    - He says, “Relax, I’ll just ask ChatGPT to fix it.”
    - Except… Cloudflare is down. The code assistants won’t load—
    - and suddenly he remembers: you can’t prompt your way out of an outage.

Why Systems Matter in the AI Era?

- From the perspective of programmer
  - ~~Vibe Coding is All You Need? “Code is cheap. Show me the talk.”~~
  - You are responsible for the program you create, whatever automation you might employ to create them
  - Human judgement remains firmly in the loop: even if or as an LLM is generating an artifact that we will use (writing, test cases, documentation, code, etc.), their output is the responsibility of the human using them.

Why Systems Matter in the AI Era?

- From the perspective of programmer

"The closer code is to the system that we ship, the greater care needs to be shown when using LLMs.

Even with something that seems natural for LLM contribution (e.g., writing tests), one should still be careful:

it's easy for LLMs to spiral into nonsense on even simple tasks."

--- "Using LLMs at Oxide"

Why Systems Matter in the AI Era?

- From the perspective of programmer
  - System is also fun, because understanding gives you control.
  - Systems are “explainable”. Everything happens for a reason.
    - A system has inputs, state, and rules
    - When something breaks, there’s usually a traceable causal chain (bug)
    - You can ask: What changed? Why now? Under what conditions?
    - Every bug leaves a trail; every crash is inevitable. If you dig deep enough, you will always find the exact root cause.

Systems are complex, but not mysterious.

Why Systems Matter in the AI Era

- If you want to BUILD better AI
  - Break through compute and memory walls with low-level system optimization

- If you want to USE AI safely
  - Debug AI-generated code by mastering a deeper mental model of systems.

"In the AI Era, code is cheap,
but understanding is priceless."

Overview

- Representing Program (Chapter 2)
- Translating Program (Chapter 3&4)
- Executing Program (Chapter 7&8&9)
  - Hardware Organization
- Memory Architecture (Chapter 6)
- Operating System
- Network

Representing Program

code/intro/hello.c

```
#include <stdio.h>

int main()
{
    printf("hello, world\n");
}
```

code/intro/hello.c

The `hello` program

Representing Program

- Source program from the computer's perspective
  - A sequence of bits (0 or 1)
  - 8-bit chunks → bytes
  - Each byte represents some text character
  - ASCII standard

#	i	n	c	l	u	d	e	<sp>	<	s	t	d	i	o	.
35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	<sp>	m	a	i	n	(	)	\n	{
104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
\n	<sp>	<sp>	<sp>	<sp>	p	r	i	n	t	f	(	"	h	e	l
10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
l	o	,	<sp>	w	o	r	l	d	\	n	"	)	;	\n	}
108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	125

What about Chinese Character?

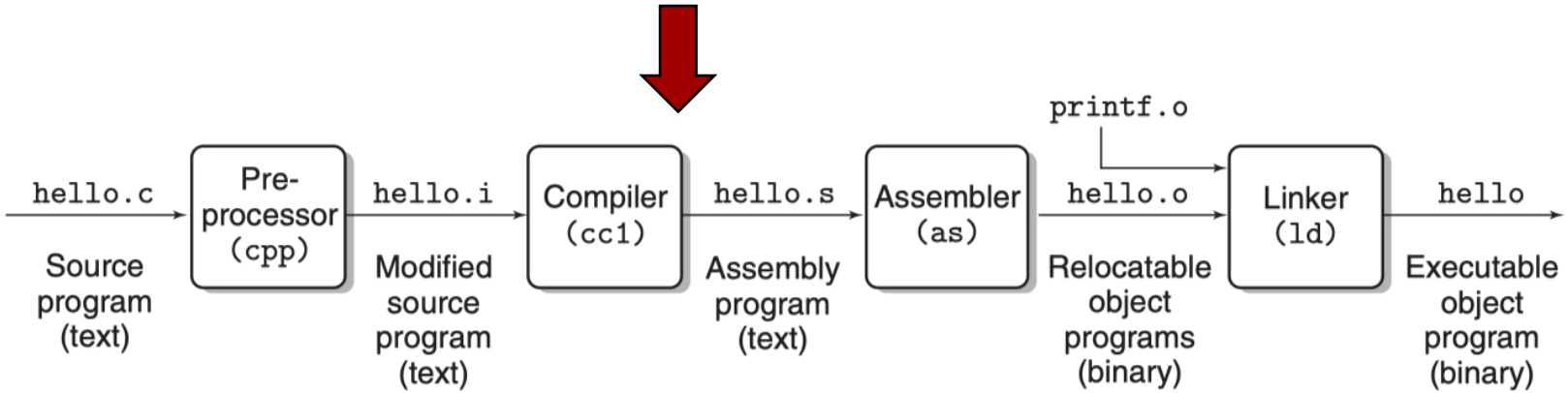
Representing Program

- Source program from the computer's perspective
  - A sequence of bits (0 or 1)
  - 8-bit chunks → bytes
  - Each byte represents some text character
  - ASCII standard
- All information in a system is represented as a bunch of bits
  - Integer, floating number, text character, …
  - How to distinguish?
    - Contexts!
- Lessons Leaned
  - As a programmer, we need understand machine representations of numbers

Translating Program

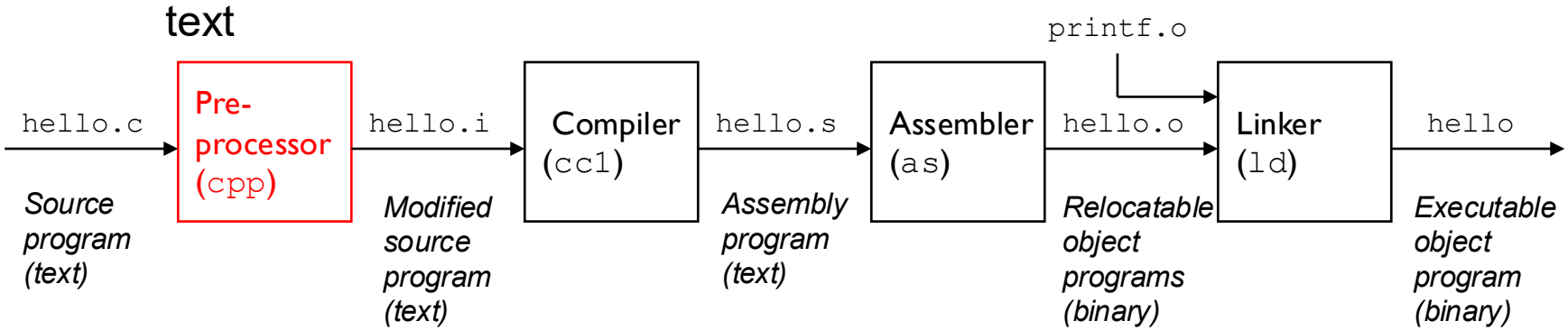
- C program is a high-level language
  - Why: Easy to be understood by human
- Machine only execute instructions (i.e., low-level *machine language*)
  - A binary disk file (called executable object files)

```
unix> gcc -o hello hello.c
```



Translating Program

- Preprocessing phase (`cpp`)
  - Modifies the original C program according to directives that begin with the # character
  - `hello.c`: Read the contents of `stdio.h` and insert it into the program text



code/intro/hello.c

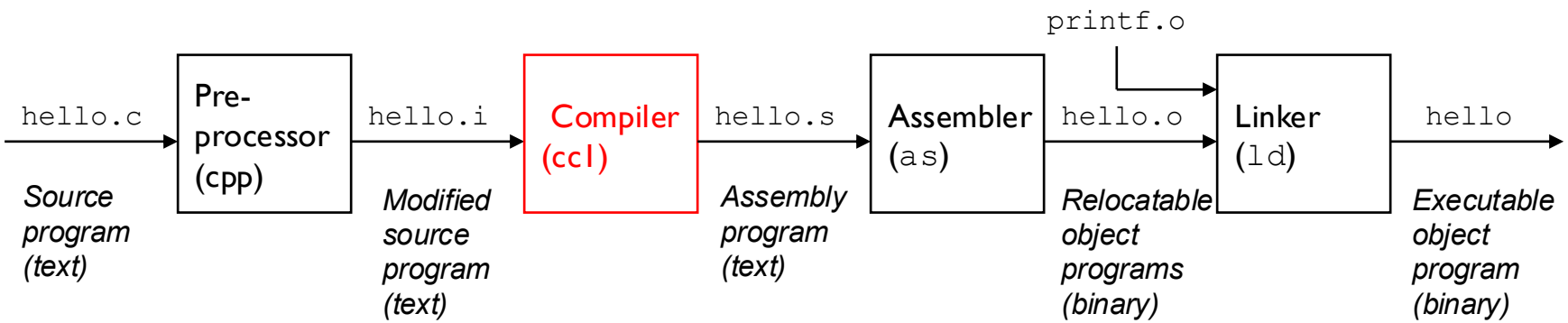
```
#include <stdio.h>

int main()
{
    printf("hello, world\n");
}
```

code/intro/hello.c

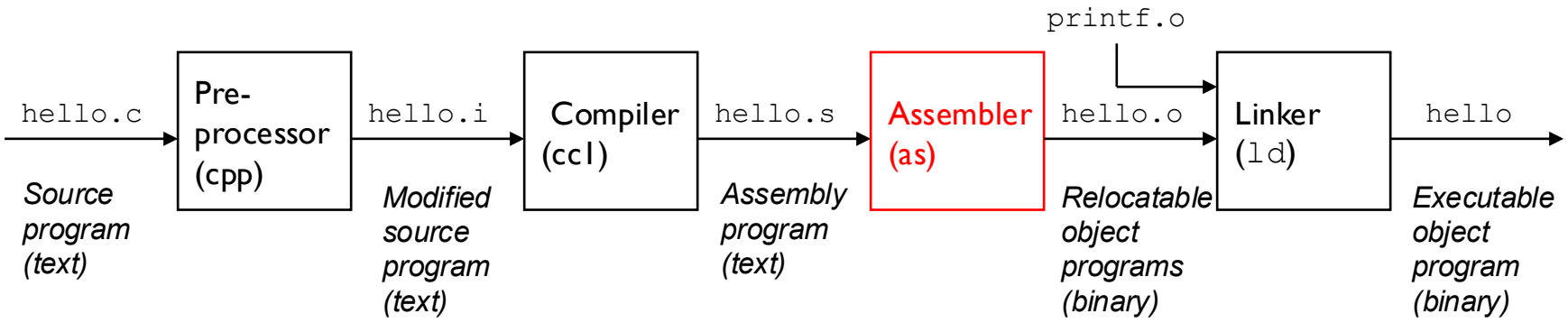
Translating Program

- Compilation phase (`cc1`)
  - Translates the C to an assembly-language program
  - Assembly-language
    - Also in a standard text form
    - Each statement exactly describes one low-level machine-language instruction



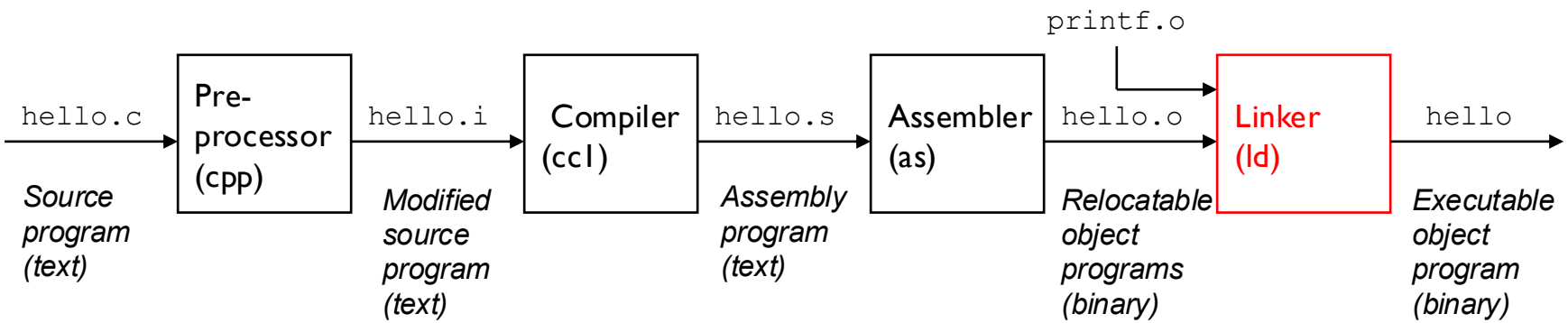
Translating Program

- Assembly phase (`as`)
  - Translates `hello.s` into machine-language instructions
  - Package into *relocatable object program*



Translating Program

- Linking phase
  - Where to find `prinf`?
    - `printf.o`
    - Provided by Standard C library
  - Merge `hello.o` and `prinf.o`
  - Result: `hello` (i.e., *executable object file*)



Why we need to understand this

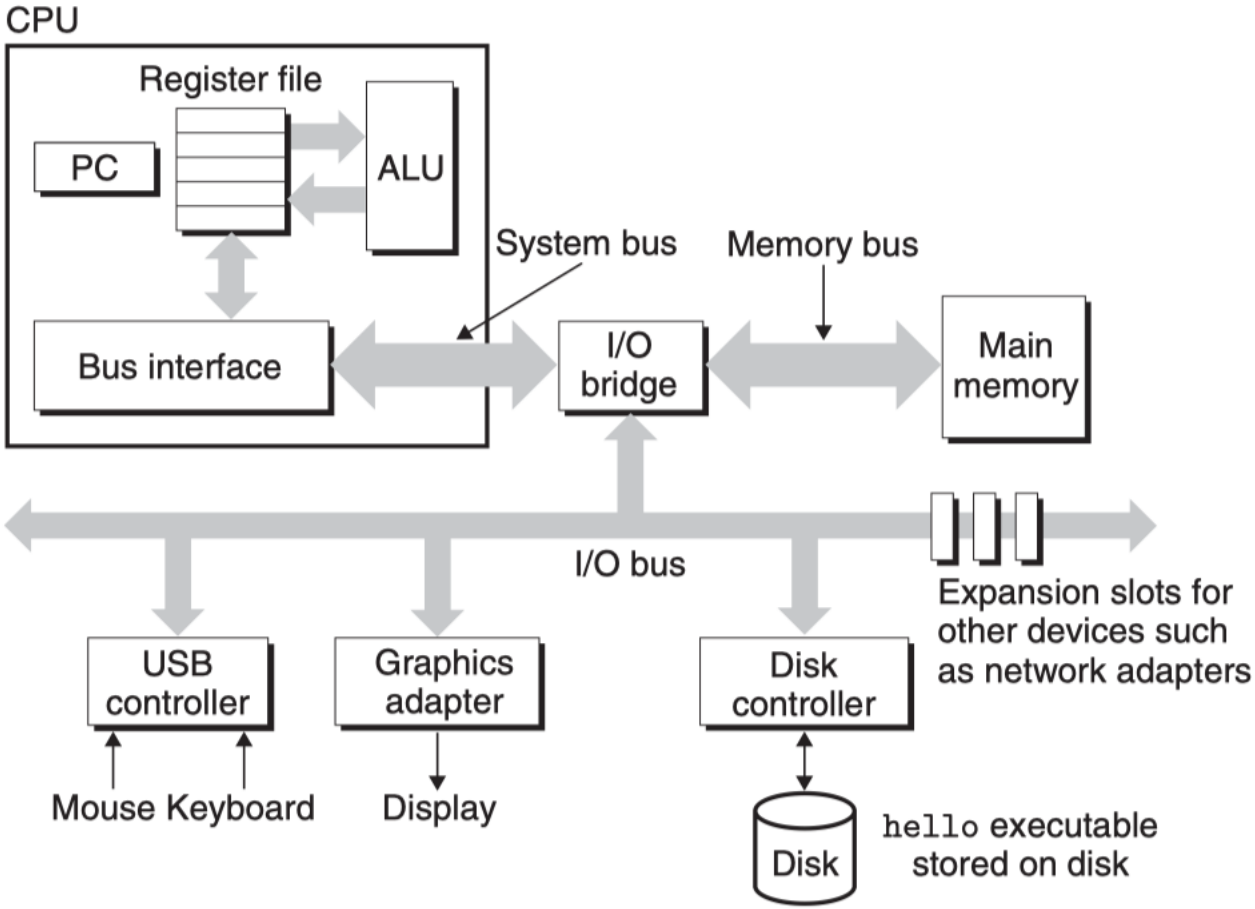
- Eliminating bugs
  - #define min(x, y) x < y ? x : y [example01.c]
- Optimizing program performance
  - If-else vs. switch-case
  - foo * 1024 → foo << 10
- Understanding link-time errors
  - undefined reference to….
- Avoiding security holes
  - Buffer overflow

Executing Program

```
unix> ./hello
hello, world
unix>
```

Shell loads and runs the program

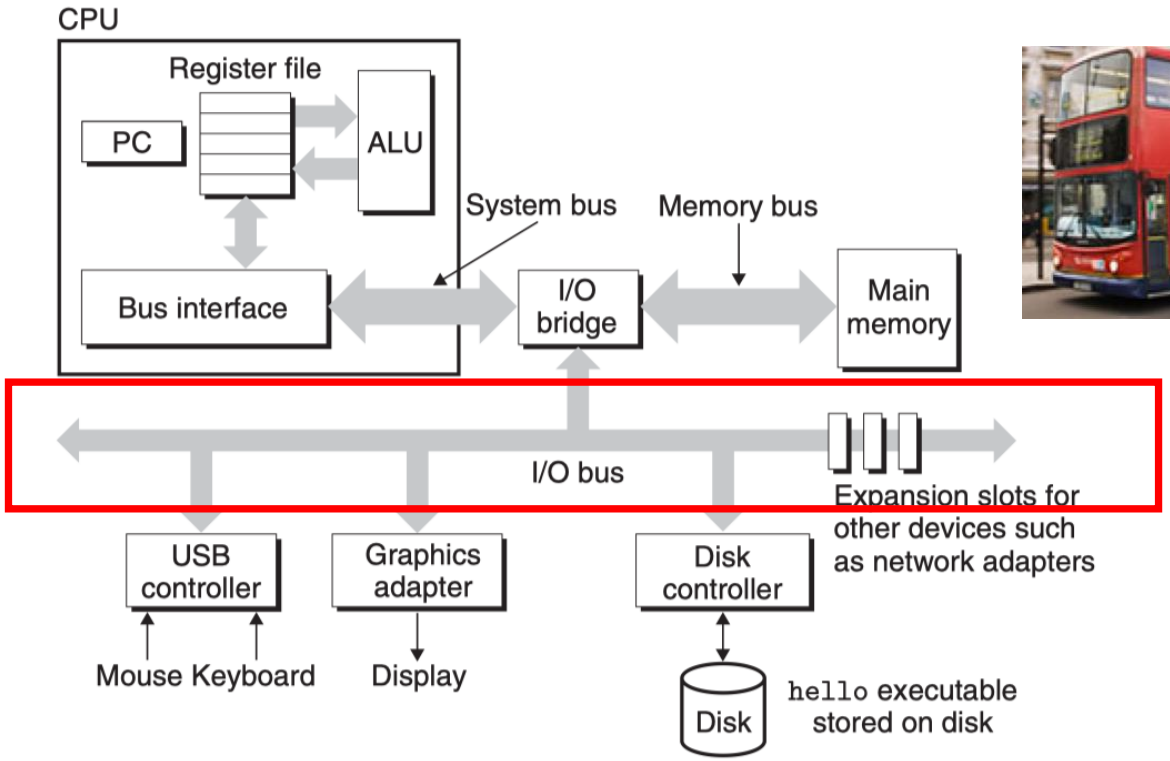
Hardware Organization



Hardware organization of a typical system

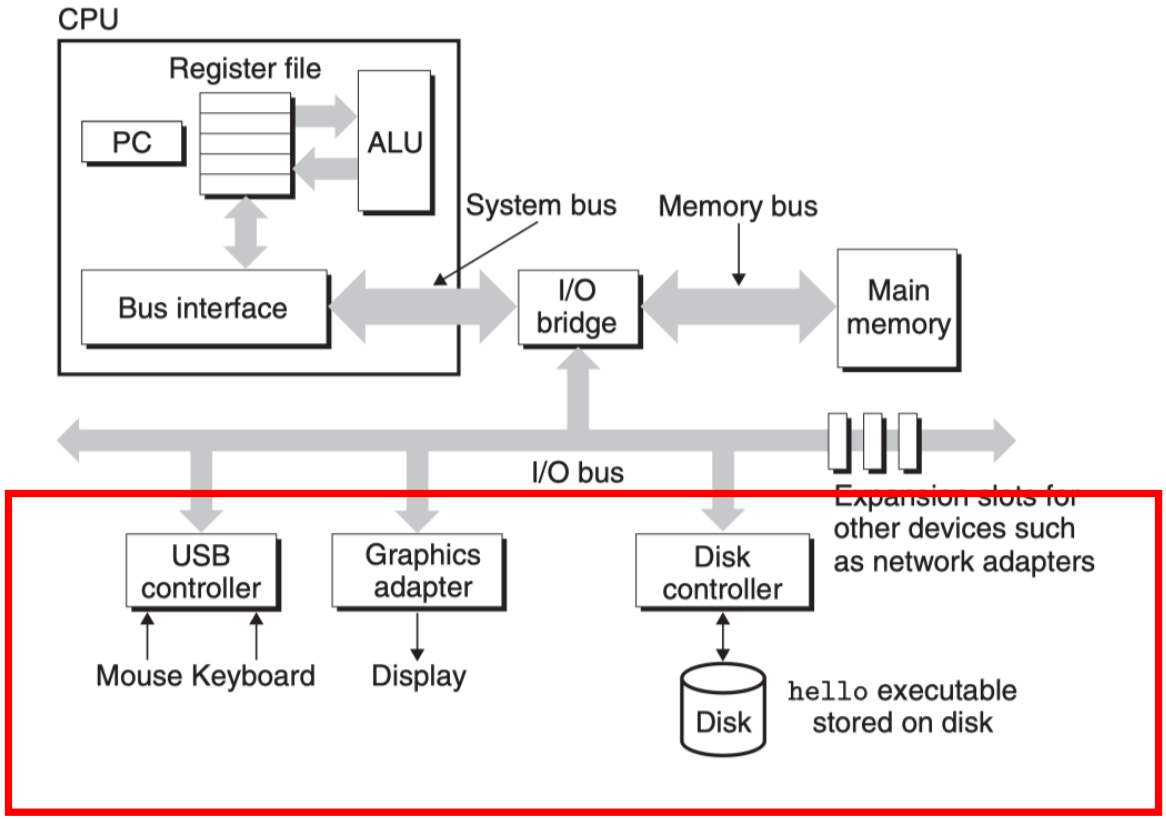
Hardware Organization

- Buses



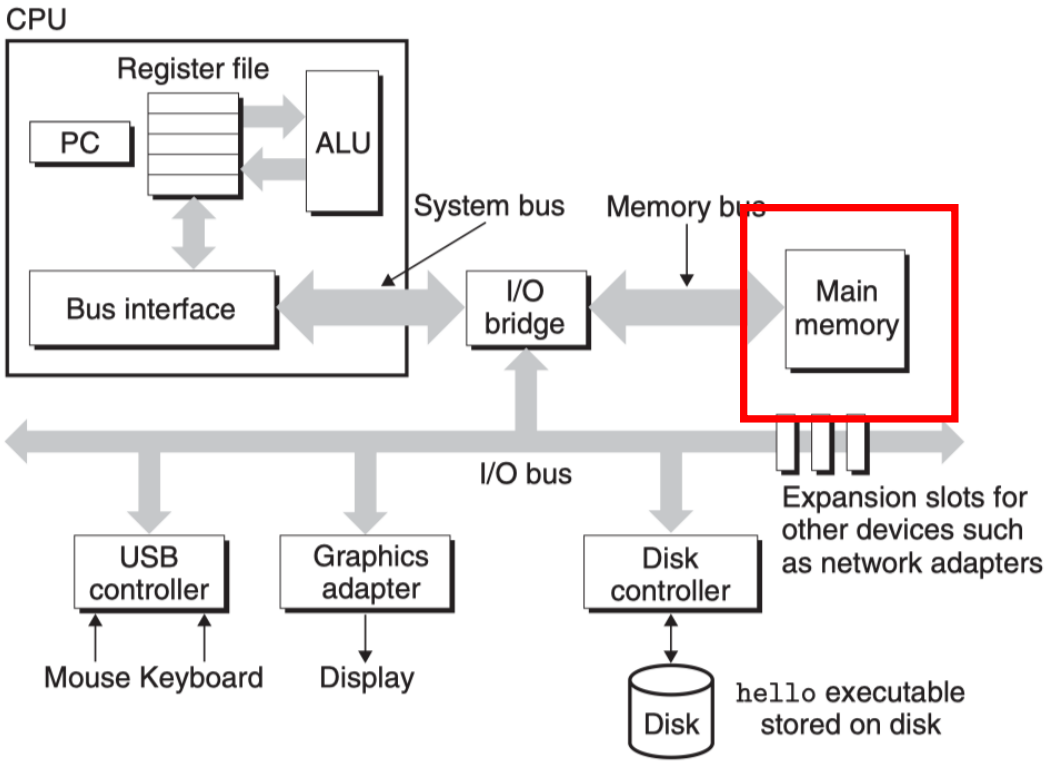
Hardware Organization

- I/O Devices (Chapter 6&10)



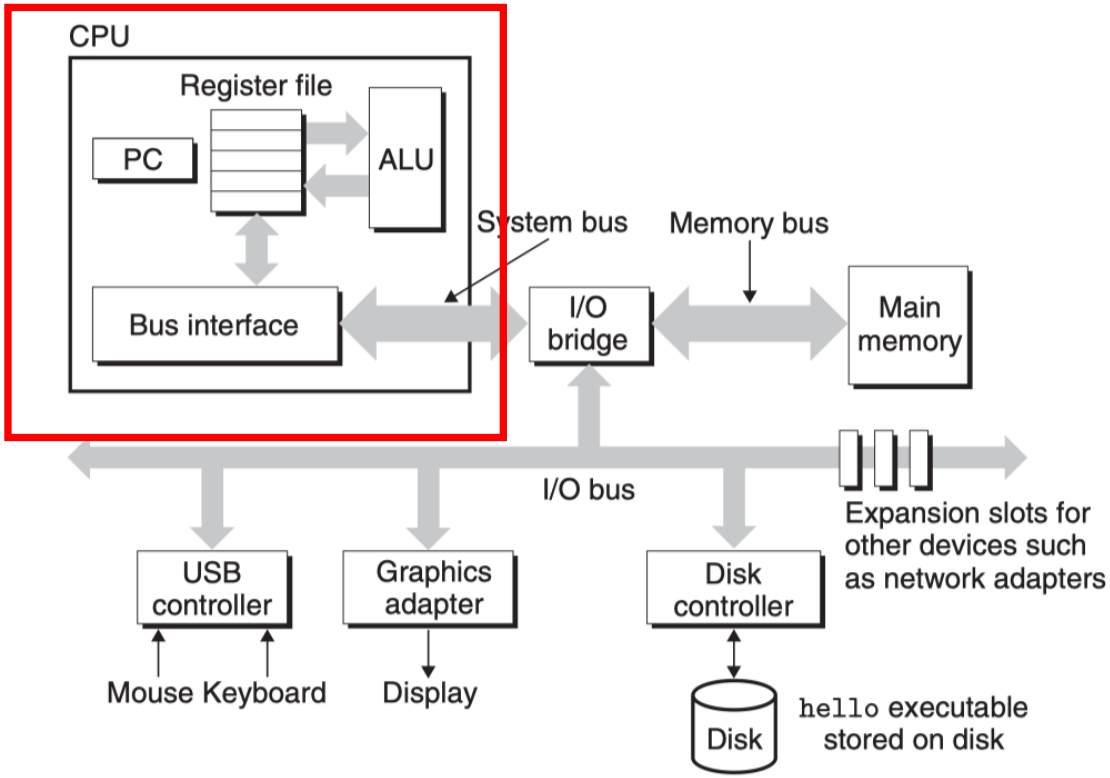
Hardware Organization

- Main Memory (Chapter 6)
  - Temporary storage



Hardware Organization

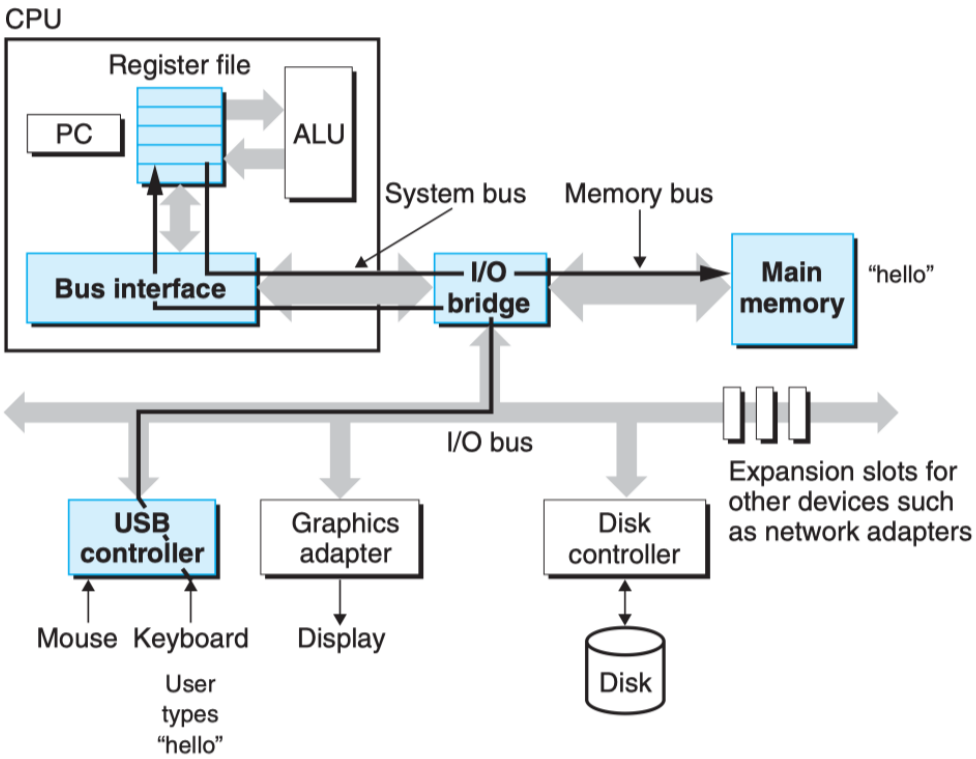
- Processor (Chapter 4)



Executing Program

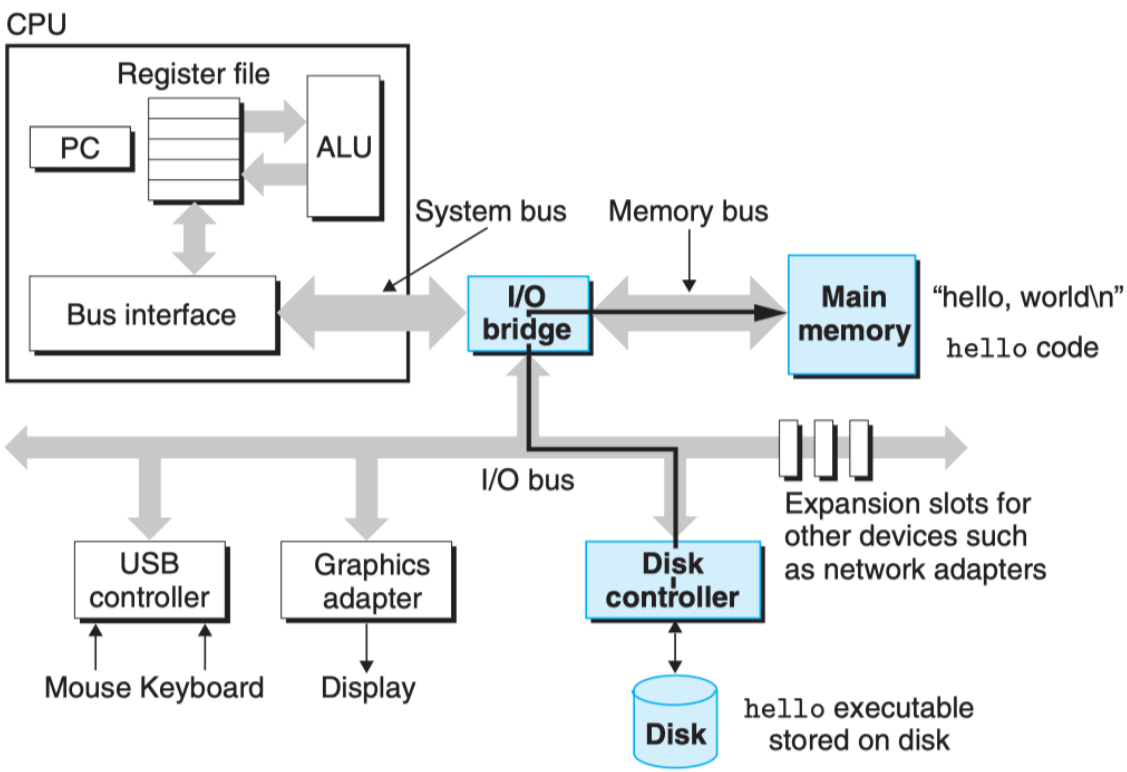
- Shell read "./hello" from keyboard into a register
- Store it into memory

```
unix> ./hello
hello, world
unix>
```



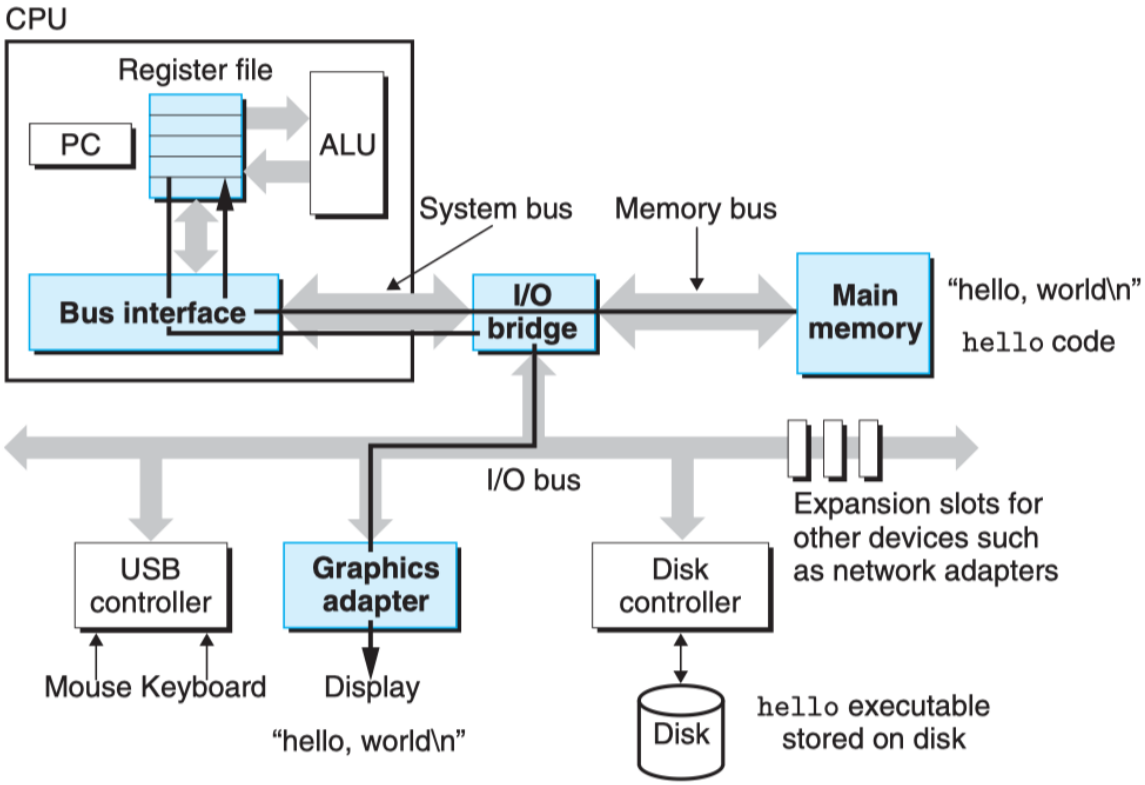
Executing Program

- Load “hello” into main memory
  - Copies the code and data from disk to main memory
  - DMA



Executing Program

- Execute the machine-language instructions
  - Copy “hello, world\n” from memory to the registers
  - Copy from registers to the display device

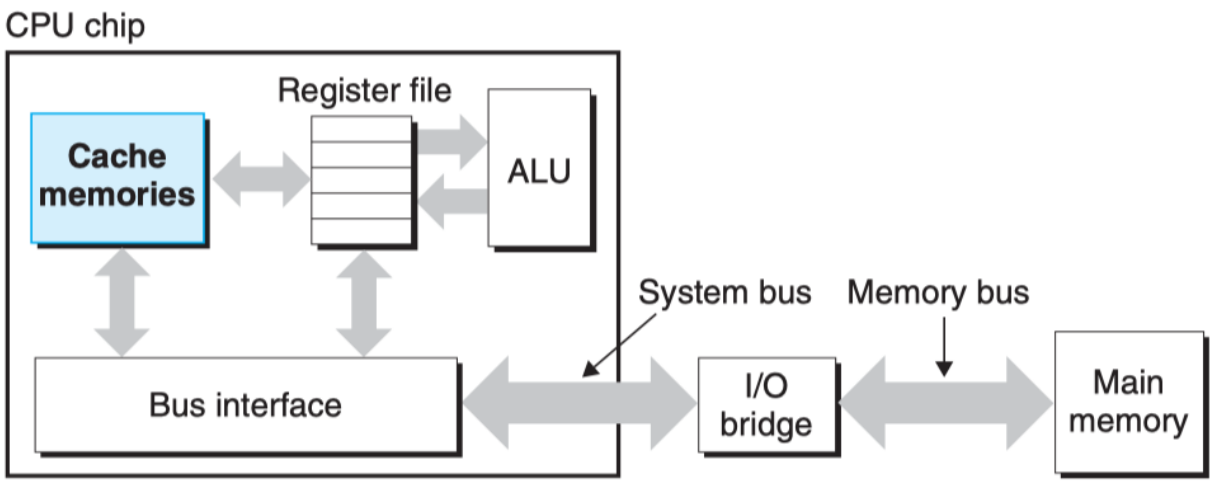


Memory Architecture

- Spends a lot of time moving information from one place to another!
- Disk vs. Main memory
  - 1,000x larger
  - 10,000,000x slower
- Registers vs. Main memory
  - 100s bytes vs. 10s GB
  - 100x faster
- Laws
  - Larger: slower
  - Faster : more expensive

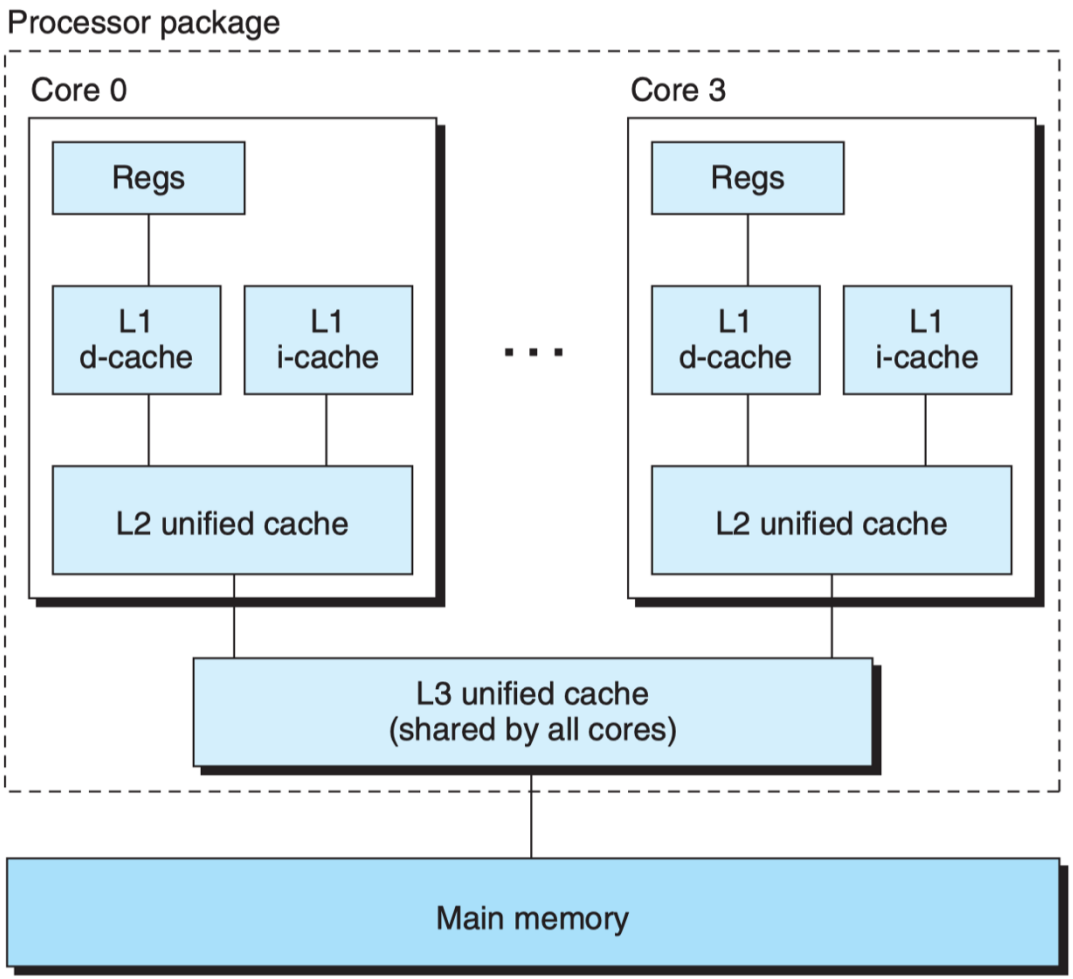
Memory Architecture

- Cache
  - SRAM
  - 10s MB (Intel i7-11700, 16MB Cache)
  - 5x slower than registers
  - 5-10x faster than main memory



Memory Architecture

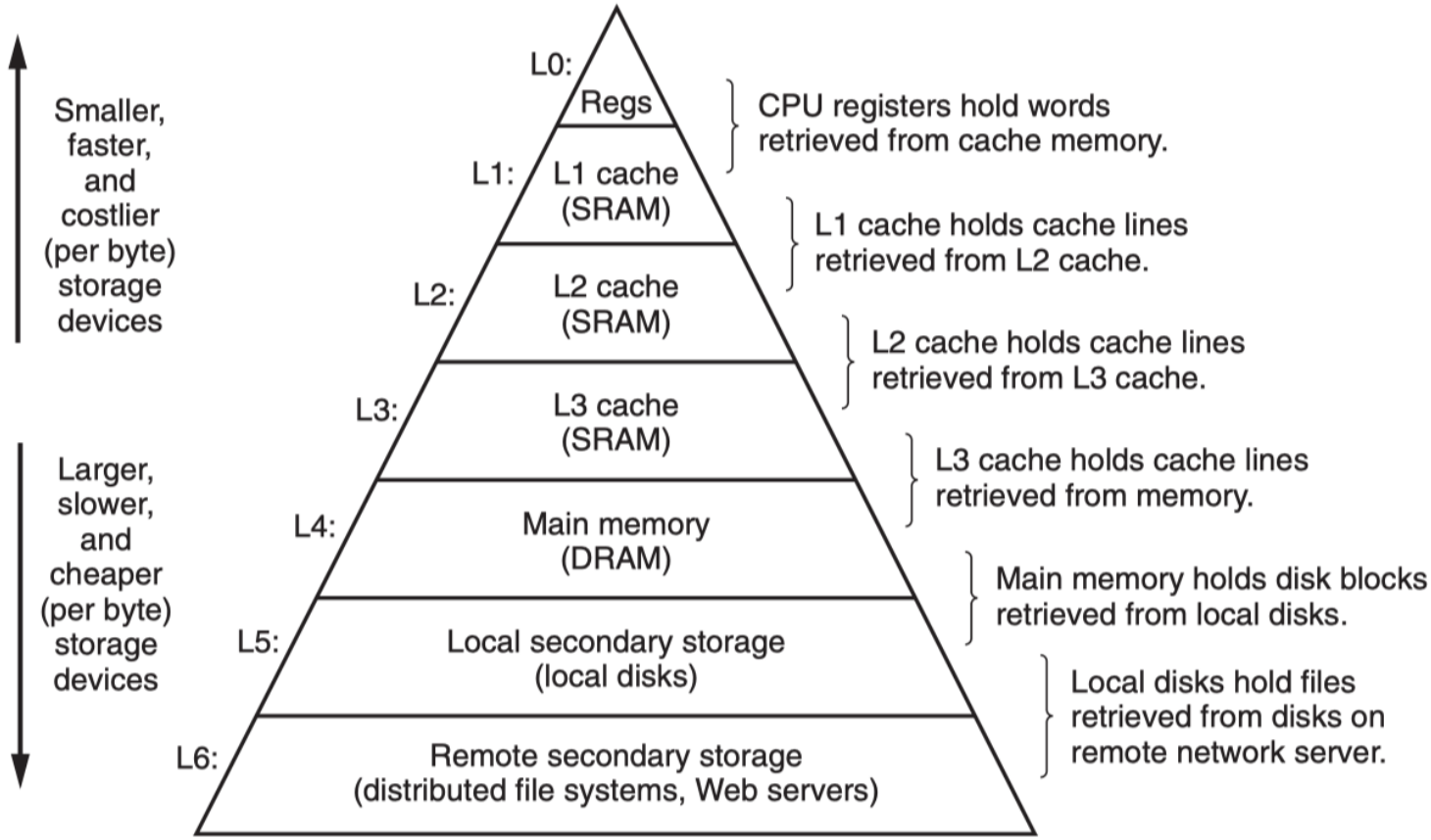
- Cache



Intel Core i7

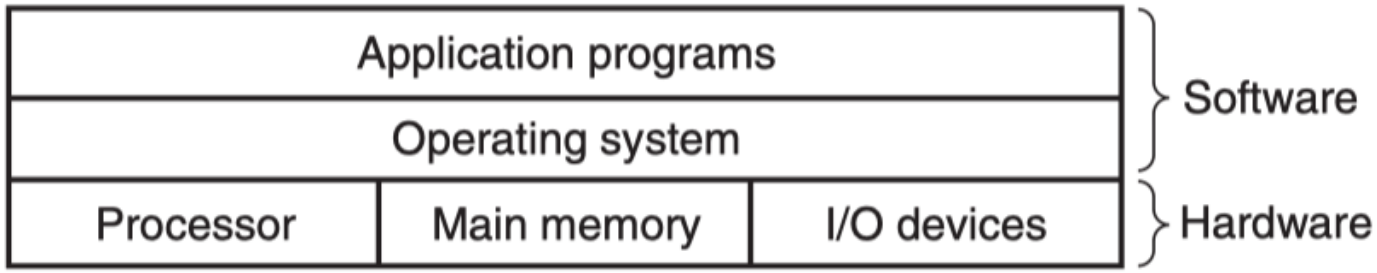
Memory Architecture

- Memory hierarchy
  - Speed → Registers/Cache
  - Size → Disks



Operating System

- A layer of software between application and hardware
  - Protect the hardware
    - Applications can be evil and vulnerable
  - Provide applications with simple and uniform mechanisms
    - Low-level hardware devices are quite different from each other



Summary

- Information = bits + context
- Programs are translated by compilers
  - From ASCII text to binary executable file
- Memory: store binary instructions
- Processor: execute binary instructions
- Memory is important
  - Computers spend most of their time copying data
  - Memory hierarchy
    - Speed: register/cache
    - Size: disk
- Operating System: managing hardware